TrustHome PH

Bank Catalog Scraper & Data Operations

Everything the trusted Python tooling does: bank-catalog scraping, scheduled maintenance, job queue, backups, restores, migrations and seed data.

Document	TH-06 · Bank Catalog Scraper & Data Operations
Version / date	Rev. 2.0 · 4 October 2026
Factual baseline	TrustHome_Current_Progress_Report_2026-10-04.md (PR #42 `fixes` working copy, based on `features/v1-3-navigation-settings`; excludes `dev`)
Replaces	System Build Plan §7 + new v2 tooling
Audience	Team Idle developers, Scrum Master, QA, panel/adviser reviewers, and AI coding assistants used by the team

Evidence label	Meaning (never use these interchangeably)
[Implemented in source]	Code exists in the repository working copy. Not proof that it runs in production.
[Unit/emulator tested]	Covered by automated unit tests or the Firestore emulator suite.
[Reported working]	A team member observed it working in the browser against the test project.
[Deployed/live verified]	Deployed to trusthome-ph and confirmed working there.
[Known gap]	Does not yet meet the product claim. Do not present it as working.
[Roadmap] / **[Proposed]**	Planned future work / a recommendation in this document, not yet built.

Contents

1. Package Overview . . . 3
2. Scraper Behaviour . . . 3
2.1 Sources . . . 3
2.2 Validation and fail-closed rule . . . 4
2.3 Upsert and delisting (`database.py`) . . . 4
3. Jobs and Schedules . . . 4
3.1 GitHub Actions practices . . . 4
4. Backup and Restore . . . 5
4.1 Backup . . . 5
4.2 Restore . . . 5
5. Migrations . . . 6
6. Demo and Seed Data . . . 6

1. Package Overview

All trusted Python tooling lives in the canonical `bank_scraper/` directory, which is what `.github/workflows/scrape.yml` runs. The v2 package (`bank_scraper_2.0`) was promoted into this path and the duplicate directories were removed. The scraping algorithm is byte-identical to the previous copy (same SHA-256 for `scraper.py`); v2 is **more complete and safer operationally, not faster**.

File	Role	Writes to Firestore?	Evidence
`scraper.py`	Fetch + parse bank sources; `--dry-run` supported	Via `database.py`	[Unit/emulator tested]
`database.py`	Deterministic IDs, merge upsert, first/last seen, delisting, keyword prefixes	Yes	[Unit/emulator tested]
`worker.py`	Polls admin-created `scraperJobs` every 15 s (trusted environment only)	Yes	[Implemented in source]
`scheduler.py`	Local loop: run once at start, then every 24 h	Yes	[Implemented in source]
`complete_expired_bookings.py`	Confirmed bookings with past end date → Completed	Yes	[Implemented in source]
`trust_scores.py`	Recompute and cache `trustScores`	Yes	[Unit/emulator tested]
`backup_firestore.py`	Export every top-level collection to JSONL	No (read only)	[Unit/emulator tested] with fakes
`restore_firestore.py`	Validate backup, dry-run by default, `--apply` writes in batches of 400	Only with `--apply`	[Unit/emulator tested] with fakes
`migrate_verification_documents.py`	Convert documents to authenticated Cloudinary delivery + metadata	Only with `--apply`	[Unit/emulator tested]; **do not apply yet**
`seed_demo_data.py`	Synthetic demo data (no real IDs, no Auth users)	Unless `--dry-run`	[Implemented in source]
`test_*.py`	28 unittest cases (5 more than the old package: backup/restore + migration)	—	[Unit/emulator tested]

2. Scraper Behaviour

2.1 Sources

Bank	Method	Status
Metrobank	Paginated JSON endpoint (not rendered HTML); page size 12; 2-second delay between requests; retries on transient errors	**Active** — default and only production source
Landbank	HTML table/card/text fallback parsers	Not active — selectors and live output must be validated first [Roadmap]
BDO	Placeholder endpoint	Not implemented — never describe as live [Roadmap]

2.2 Validation and fail-closed rule

- A record is kept only if it has a **reference number** and a **finite price greater than 1**.
- If a source returns **zero valid records**, the run **fails** instead of treating the empty result as "everything was sold" — this prevents a site outage or layout change from delisting the whole catalog.

2.3 Upsert and delisting (`database.py`)

```
doc_id = f"{bank}_{reference_no}"                      # deterministic -> idempotent re-runs, no duplicates
db.collection("bankProperties").document(doc_id).set({
    **item, "bank": bank, "status": "active",
    "lastSeen": SERVER_TIMESTAMP,                       # firstSeen set only when the doc is new
    "keywords": location_prefixes(item["location"]),    # lowercase prefixes for array-contains search
}, merge=True)

# afterwards: previously active docs for this bank that were not seen in THIS successful run
#   -> status "delisted", lastSeen updated. History is kept; nothing is deleted.
```

BEST PRACTICE · Responsible scraping

- Prefer official JSON/feeds over HTML parsing (as with Metrobank); respect `robots.txt` and the site's terms; identify the client with a descriptive User-Agent.
- Keep the polite delay and retry with back-off; never parallelise requests to the same bank.
- Store only listing facts and the source link; show "last seen" and link back to the bank as the authority.
- Describe the catalog as a curated public scrape, not a partnership or an official bank channel.
- Before enabling a new source: save sample responses as test fixtures, add parser tests, run `--dry-run`, compare 10 records manually against the site, then enable.

3. Jobs and Schedules

Trigger	What runs	Cadence	Status
GitHub Actions `scrape.yml`	scrape → complete expired bookings → recompute trust scores	Weekly (cron) + manual dispatch	**[Implemented in source]** — validate secret access and a controlled run
Admin "queue refresh" (`/admin/bank-catalog`)	Creates `scraperJobs/{id}`; `worker.py` picks it up	Only while a trusted worker is running	**[Implemented in source]**
Local `scheduler.py`	Scrape	At start, then every 24 h	**[Implemented in source]**
Firebase scheduled functions	Daily completion / trust	—	Source only, not deployed

CAUTION · Implications of weekly cadence

- Expired Confirmed bookings may wait up to a week to become Completed (unless the renter completes them) — so rating prompts and trust scores lag. Document this, or shorten the cron (e.g. daily) if Actions minutes allow.
- An admin-queued job does nothing unless a trusted worker is running; the admin UI should show the job as queued, not done.

3.1 GitHub Actions practices

- Secret `FIREBASE_SERVICE_ACCOUNT` holds the key JSON; the workflow writes it to a temp file with restrictive permissions and removes it at the end. Never `echo` the secret or `cat` the file.

- Pin action versions (e.g. `actions/checkout@v4`, `actions/setup-python@v5`) and the Python version; cache pip.
- Use `concurrency:` so two runs never overlap, and `timeout-minutes:` so a hung bank site cannot burn minutes.
- Run steps with `set -e` semantics: if scraping fails, still run expired-booking completion and trust scores? Decide explicitly — currently they run in sequence; prefer independent steps with `if: always()` for the maintenance jobs so a bank outage does not block them.
- Add `workflow_dispatch` for manual runs and record each run's result in the release log during QA.
- Watch Firestore quota: a full rescore reads every verified listing plus its bookings/ratings.

4. Backup and Restore

4.1 Backup

```
python backup_firestore.py                      # every top-level collection -> timestamped JSONL
                                                # default location: under your home directory
```

- Firestore timestamps are preserved; files are written through a temp file and existing backups are **never overwritten**.
- Backups contain personal data (names, emails, addresses, document references). Store them encrypted, outside the repository and outside synced folders; delete old backups on a schedule.
- Subcollections are not covered if any are introduced later — extend the script first.

4.2 Restore

```
python restore_firestore.py <backup-dir>          # validates the WHOLE backup, dry run (no writes)
python restore_firestore.py <backup-dir> --apply  # writes in batches of 400
```

CAUTION · Restore semantics

- Restore upserts documents from the backup; it does **not** delete documents that are absent from the backup.
- Never rehearse `--apply` against production. Rehearse against the emulator or a separate project first; no live rehearsal is recorded yet.

BEST PRACTICE · Backup routine

- Take and verify a backup (record counts per collection) before every migration, bulk seed, or rules change that rewrites data.
- 3-2-1 idea scaled down: at least two copies, one off the laptop (encrypted cloud drive with restricted sharing).
- **[Proposed]** On Blaze, enable Firestore managed exports / point-in-time recovery instead of relying only on scripts.

5. Migrations

Migration	Purpose	State	Prerequisites
Address migration	Move legacy public `address` fields into `listingPrivate`	Review existing script output; apply only with backup + dry run + rollback plan	Verified backup; rules that already forbid public address
`migrate_verification_documents.py`	Rename eligible Cloudinary assets to authenticated delivery, write `listingPrivate.documents` metadata, remove old URL fields only on success	**Do not apply to live**	App must first generate and consume short-lived authenticated URLs (TH-04 §6)

1. Freeze writes to the affected collection (announce a maintenance window).
2. Backup → verify counts.
3. Dry run → save output → peer review the counts, conflicts and skipped records.
4. `--apply` → verify in console and in the app → re-run to prove idempotency (second run changes nothing).
5. Deploy any dependent rules/frontend; record the migration in the release log.

6. Demo and Seed Data

- `seed_demo_data.py --dry-run` first, review counts, then run. It creates synthetic records only — no real IDs, passwords or Auth users; placeholder documents are labelled synthetic.
- Seed enough verified listings across several cities and types for fairness comparables; enough completed bookings and ratings that trust ordering visibly differs from price/recency order; one founded dispute for the public flag; a pending listing for the review queue.
- Run a Metrobank scrape shortly before the demo so “last seen” is recent.
- Keep demo data clearly distinguishable from tester data and remove it after the defense.